

PROYECTO FINAL - PRIMERA ENTREGA

KineSys

Documento enfocado en los 3 módulos trabajados

Pacientes - Terapeutas - Terapias y Tratamientos

Estudiantes:

Eckard Antony Rodriguez Gutierrez

Luis Alejandro Narvaez Teran

Docente: Maribel Romero Mestre

Universidad Popular del Cesar - Sede Sabanas

Ingeniería de Sistemas

Valledupar - Cesar

2026

Versión corregida el 26 de abril de 2026. Se limita a los tres módulos que venían desarrollados en el PDF original y agrega las pruebas automatizadas correspondientes.

Tabla De Contenido

1. Descripción del sistema para los tres módulos	2.1 Introducción
1.1 Identificación del problema	2.2 Planificación
1.2 Descripción de los módulos	2.3 Pruebas unitarias por módulo
1.3 Requisitos funcionales y no funcionales	2.4 Pruebas de integración
1.4 Casos de uso	2.5 Pruebas de sistema
1.5 Diseño del sistema	2.6 Pruebas de aceptación
1.6 Producto del software	3. Conclusiones
2. Pruebas del software	4. Referencias y pendientes

1. Descripción Del Sistema Para Los Tres Módulos

1.1 Identificación Del Problema

La clínica de fisioterapia necesita controlar tres procesos centrales: el registro de pacientes, el registro de terapeutas y la gestión de terapias/tratamientos disponibles. Si estos procesos se gestionan manualmente, se producen errores de digitación, duplicidad de documentos, terapeutas sin información profesional completa, tratamientos sin especialidad definida y dificultad para relacionar equipos con terapias.

El alcance de esta entrega se concentra únicamente en esos tres módulos, porque son los módulos que venían desarrollados en el PDF original y sobre los que se diseñan clases de equivalencia, valores límite, caminos básicos, pruebas unitarias, integración y aceptación.

1.2 Descripción De Los Módulos

Registro y Gestión de Pacientes

Módulo encargado de registrar, consultar, actualizar y eliminar pacientes. Gestiona datos personales, documento, contacto, EPS, seguro médico, afiliación y rol Paciente.

Archivos: PatientForm.jsx, PatientList.jsx, pacientesService.js, pacientesApi.js, Paciente.cs, PersonasController/PacientesController.

Registro y Gestión de Terapeutas

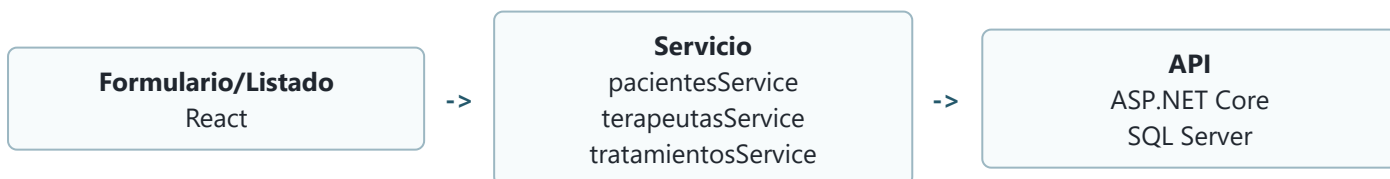
Módulo encargado de registrar, consultar, actualizar y eliminar terapeutas. Administra información personal, acceso, contacto, licencia profesional, título académico, años de experiencia, fecha de contratación, estado y especialidades.

Archivos: TerapeutaForm.jsx, TerapeutasList.jsx, terapeutasService.js, terapeutasApi.js, Terapeuta.cs, PersonasController/TerapeutasController.

Gestión de Terapias y Tratamientos

Módulo encargado de registrar, consultar, actualizar y eliminar tratamientos. Maneja nombre, descripción, especialidad, duración, costo, sesiones recomendadas, frecuencia, estado, indicaciones, contraindicaciones y equipos requeridos.

Archivos: TratamientoForm.jsx, TratamientosList.jsx, tratamientosService.js, tratamientosApi.js, Tratamiento.cs, TratamientosController.



1.3 Modelo De Requerimientos

Requisitos Funcionales

ID	Módulo	Descripción
RF-PAC-01	Pacientes	Registrar pacientes con datos personales, documento, contacto, EPS, seguro y rol Paciente.
RF-PAC-02	Pacientes	Listar únicamente personas con rol Paciente (rol = 1).
RF-PAC-03	Pacientes	Consultar un paciente por identificador y rechazar registros que no tengan rol Paciente.
RF-PAC-04	Pacientes	Actualizar datos del paciente mediante endpoint /Update/{id}.
RF-PAC-05	Pacientes	Eliminar paciente por identificador propagando errores funcionales del API.
RF-TER-01	Terapeutas	Registrar terapeutas con rol Terapeuta (rol = 2), datos de acceso y datos profesionales.
RF-TER-02	Terapeutas	Listar únicamente personas con rol Terapeuta (rol = 2).
RF-TER-03	Terapeutas	Consultar un terapeuta por identificador y rechazar registros que no tengan rol Terapeuta.
RF-TER-04	Terapeutas	Actualizar terapeuta preservando rol 2 y normalizando datos profesionales.
RF-TER-05	Terapeutas	Obtener únicamente terapeutas activos.
RF-TRA-01	Tratamientos	Registrar tratamientos asociados a una especialidad válida.
RF-TRA-02	Tratamientos	Listar, consultar, actualizar y eliminar tratamientos.
RF-TRA-03	Tratamientos	Validar nombre obligatorio, especialidad, duración, sesiones, costo y frecuencia.
RF-TRA-04	Tratamientos	Agregar y remover equipos requeridos para un tratamiento mediante rutas anidadas.
RF-TRA-05	Tratamientos	Advertir cuando la cantidad de equipo solicitada supera la disponibilidad real.

Requisitos No Funcionales

ID	Atributo	Descripción
RNF-01	Mantenibilidad	Cada módulo separa UI, servicio y API client para facilitar pruebas y cambios controlados.
RNF-02	Seguridad	Las operaciones se ejecutan bajo sesión autenticada; el token JWT lo agrega apiCliente mediante Authorization Bearer.
RNF-03	Integridad	Paciente y Terapeuta se distinguen por rol; Tratamiento exige especialidad válida y relaciones con equipos.
RNF-04	Usabilidad	Formularios con campos requeridos, listas, búsquedas, estados de carga y mensajes de error.
RNF-05	Portabilidad	Frontend React/Vite ejecutable en navegador moderno; API .NET 8 con SQL Server.
RNF-06	Testabilidad	Servicios principales cubiertos con Vitest/Jest, mocks de APIs y salidas de ejecución guardadas.

1.4 Modelo De Casos De Uso

Actor principal para los tres módulos: Administrador. Los módulos se usan desde el panel administrativo para mantener la información base de la clínica.

ID	Actor	Caso de uso	Precondición	Resultado esperado
CU-PAC-01	Administrador	Registrar paciente	Formulario con datos válidos	Paciente creado con rol 1 y credenciales iniciales por documento.
CU-PAC-02	Administrador	Actualizar paciente	Paciente existente	Datos actualizados en backend.
CU-PAC-03	Administrador	Eliminar paciente	Paciente seleccionado	Paciente eliminado o error mostrado si la operación falla.
CU-TER-01	Administrador	Registrar terapeuta	Formulario con datos personales y profesionales válidos	Terapeuta creado con rol 2 y activo.
CU-TER-02	Administrador	Actualizar terapeuta	Terapeuta existente	Datos actualizados preservando rol Terapeuta.
CU-TER-03	Administrador	Listar terapeutas activos	Personas cargadas desde API	Se muestran solo terapeutas activos.
CU-TRA-01	Administrador	Registrar tratamiento	Especialidad seleccionada y datos válidos	Tratamiento creado con configuración clínica y económica.
CU-TRA-02	Administrador	Agregar equipo a tratamiento	Equipo y cantidad seleccionados	Equipo asociado al tratamiento o advertencia por exceso.
CU-TRA-03	Administrador	Eliminar tratamiento	Tratamiento seleccionado	Tratamiento eliminado o error mostrado si está relacionado.

1.5 Modelo De Diseño Del Sistema

Elemento	Tipo	Responsabilidad
Paciente	Persona	Hereda datos comunes y agrega EPS, seguro, afiliación, estado clínico, citas, tratamientos e historial.
Terapeuta	Persona	Hereda datos comunes y agrega licencia, título, experiencia, contratación, citas, disponibilidad y especialidades.
Tratamiento	Entidad clínica	Se relaciona con terapeuta, paciente/planes, protocolos, tipos de terapia y equipos requeridos.
pacientesService	Servicio frontend	Filtra rol 1, crea con /Register, actualiza con /Update/{id}, elimina por id.
terapeutasService	Servicio frontend	Filtra rol 2, crea con /Register, preserva rol 2 al actualizar y obtiene activos.
tratamientosService	Servicio frontend	CRUD de tratamientos y rutas /{idTratamiento}/equipos para equipos requeridos.

1.6 Producto Del Software

Elemento	Detalle
Aplicación	KineSys - Sistema de Gestión de Clínica de Fisioterapia
Módulos incluidos en este documento	Pacientes, Terapeutas, Terapias/Tratamientos
Frontend	cd kinesys; npm run dev
Pruebas	cd kinesys; npm run test / npm run test:jest / npm run test:coverage
Build	cd kinesys; npm run build
Backend	cd api/ApiPrueba; dotnet run

2. Pruebas Del Software

2.1 Introducción

Esta sección completa la plantilla de pruebas exclusivamente para los tres módulos trabajados: pacientes, terapeutas y tratamientos. Se incluyen técnicas de caja negra y caja blanca, diseño de casos, ejecución automatizada con Jest/Vitest y evaluación de resultados.

3

Módulos evaluados

33

Pruebas específicas

107

Pruebas frontend totales

10/10

Suites aprobadas

2.2 Planificación De Las Pruebas

Objetivo General

Verificar y validar que los módulos de Pacientes, Terapeutas y Terapias/Tratamientos permiten registrar, consultar, actualizar y eliminar información de forma consistente, con validación de datos, roles correctos y comunicación adecuada con los servicios/API.

Objetivos Específicos

- Verificar que el módulo de Pacientes filtre rol 1, cree credenciales iniciales por documento y propague errores del API.
- Validar que el módulo de Terapeutas filtre rol 2, preserve el rol al actualizar y maneje datos profesionales obligatorios.
- Comprobar que el módulo de Tratamientos valide especialidad, duración, costo, sesiones, frecuencia y equipos asociados.
- Ejecutar pruebas unitarias automatizadas con Vitest y Jest para los servicios de los tres módulos.
- Diseñar casos de aceptación que permitan ejecutar manualmente los flujos de interfaz pendientes.

Alcance

Módulo	Funcionalidades sometidas a prueba
Pacientes	Registro, consulta, edición, eliminación y validación de rol Paciente
Terapeutas	Registro, consulta, edición, eliminación, activos y validación de rol Terapeuta
Tratamientos	Registro, consulta, edición, eliminación, especialidad, duración, costo, sesiones y equipos requeridos

Estrategia

Nivel	Técnica	Modelo / tipo	Herramienta
Unitarias	Caja negra y caja blanca	Clases de equivalencia, valores límite, caminos básicos y mocks de API	Vitest, Jest
Integración	Incremental	Formulario -> servicio -> API client -> endpoint; validación de rol y payload	Vitest/Jest, revisión de código
Sistema	Caja negra	Build de producción y revisión de comportamiento por módulo	Vite build
Aceptación	Caja negra	Escenarios funcionales por módulo con pasos de negocio	Manual; pendiente Selenium/Katalon

Ambiente

Equipo	Especificación
Equipo de pruebas	Windows 11 Home Single Language 64 bits, Intel Core i5-1235U, 10 núcleos, 12 procesadores lógicos, 15.7 GB RAM
Repositorio	C:\Users\carde\Documents\ProgramacionMovil\KineSys

Software	Versión / Estado
Node.js	v24.14.0
npm	11.9.0
React	19.1.1
Vite	7.x; build ejecutado correctamente
Vitest	4.1.5
Jest	30.3.0
Testing Library React	16.3.2
.NET	API objetivo net8.0; SDK local 10.0.103
SQL Server	Configurado en docker-compose.yml

2.3 Pruebas Unitarias Por Módulo

Resumen De Pruebas Automatizadas

Archivo	Módulo	Casos	Cobertura funcional	Resultado
pacientesService.test.js	Pacientes	12	listar, obtener, crear, actualizar, eliminar, errores de API	Aprobado
terapeutasService.test.js	Terapeutas	12	listar, obtener, crear, actualizar, eliminar, activos	Aprobado
tratamientosService.test.js	Tratamientos	9	listar, obtener, crear, actualizar, eliminar, equipos, error 409	Aprobado

Área	Sentencias	Ramas	Funciones	Líneas
Suite completa frontend	71.83%	70.70%	76.66%	71.75%
pacientesService.js	79.31%	33.33%	100.00%	78.57%
terapeutasService.js	60.97%	71.42%	80.00%	58.97%
tratamientosService.js	74.41%	50.00%	100.00%	74.41%

2.3.1 Módulo Pacientes - Clases De Equivalencia

Campo	Clases válidas	Clases inválidas
Nombres	2 a 50 caracteres, solo letras y espacios	Vacío, menor a 2, mayor a 50, contiene números
Apellidos	2 a 50 caracteres, solo letras y espacios	Vacío, mayor a 50, contiene números
Tipo documento	CC, TI, CE, PA	Sin selección o valor no permitido
Documento identidad	6 a 15 dígitos	Vacío, menor a 6, mayor a 15, contiene letras
Correo	Formato usuario@dominio	Sin @, sin dominio, vacío si obligatorio
Teléfono/Celular	10 dígitos numéricos	Menor a 10, mayor a 10, contiene letras
Fecha nacimiento	Fecha válida pasada	Fecha futura, formato inválido
EPS/Seguro	Registro existente o vacío permitido según flujo	ID inexistente o afiliación incompleta cuando se selecciona EPS

Casos De Prueba - Pacientes

ID	Caso	Datos	Resultado esperado	Resultado obtenido
PAC-CP01	Datos completos válidos	Ana Pérez, CC, 12345678, correo válido	Registro exitoso	Aprobado por servicio
PAC-CP02	Nombre vacío	nombres = ""	Formulario bloquea y muestra alerta	Diseñado
PAC-CP03	Documento corto	12345	Error de validación	Diseñado
PAC-CP04	Persona con rol distinto	API devuelve rol 2	obtenerPaciente lanza error	Automatizado aprobado
PAC-CP05	API sin pacientes	Lista con roles 2 y 3	listarPacientes devuelve []	Automatizado aprobado
PAC-CP06	Error de eliminación	API rechaza delete	Error propagado	Automatizado aprobado

2.3.2 Módulo Terapeutas - Clases De Equivalencia

Campo	Clases válidas	Clases inválidas
Usuario	4 a 20 caracteres alfanuméricos	Vacío, menor a 4, mayor a 20, caracteres especiales
Contraseña	Mínimo 6 caracteres en creación	Vacía en creación, menor a 6
Documento identidad	6 a 15 dígitos	Vacío, fuera de rango, contiene letras
Nombres/Apellidos	2 a 50 caracteres	Vacío, menor a 2, mayor a 50, caracteres inválidos
Licencia profesional	Texto no vacío	Vacía o duplicada
Título académico	Texto no vacío	Vacío o demasiado corto
Años experiencia	0 a 99	Negativo, mayor a 99, texto
Fecha contratación	Fecha válida	Formato inválido

Casos De Prueba - Terapeutas

ID	Caso	Datos	Resultado esperado	Resultado obtenido
TER-CP01	Datos válidos	usuario, password, documento, licencia y título	Registro exitoso con rol 2	Aprobado por servicio
TER-CP02	Contraseña vacía en creación	password = ""	Formulario bloquea	Diseñado
TER-CP03	Campos profesionales vacíos	noLicencia/tituloAcademico vacíos	Formulario bloquea	Diseñado
TER-CP04	Persona con rol distinto	API devuelve rol 1	obtenerTerapeuta lanza error	Automatizado aprobado
TER-CP05	Actualizar terapeuta	datos parciales	Se preserva rol 2	Automatizado aprobado
TER-CP06	Listar activos	mezcla activo true/false	Devuelve solo terapeutas activos	Automatizado aprobado

2.3.3 Módulo Terapias/Tratamientos - Clases De Equivalencia

Campo	Clases válidas	Clases inválidas
Nombre	Texto no vacío	Vacío o solo espacios
Especialidad	ID diferente de 0 y existente	0, vacío o especialidad inexistente
Duración	5 a 240 minutos	Menor a 5, mayor a 240, no numérico
Costo base	Mayor o igual a 0	Negativo o no numérico
Sesiones recomendadas	1 a 50	0, negativo, mayor a 50
Frecuencia	Diaria, Semanal, Quincenal, Mensual	Valor diferente a los permitidos
Equipos	Equipo existente y cantidad mayor o igual a 1	Sin equipo, cantidad 0 o mayor a disponibilidad sin confirmación

Casos De Prueba - Tratamientos

ID	Caso	Datos	Resultado esperado	Resultado obtenido
TRA-CP01	Datos válidos	nombre, especialidad 1, duración 60, costo 150000	Tratamiento creado	Automatizado servicio
TRA-CP02	Nombre vacío	nombre = ""	Formulario bloquea	Diseñado
TRA-CP03	Especialidad no seleccionada	idEspecialidad = 0	Muestra error de especialidad	Diseñado
TRA-CP04	Duración menor a 5	duracionMinutos = 4	Input no permite / error	Diseñado
TRA-CP05	Costo negativo	costoBase = -1	Input no permite / error	Diseñado
TRA-CP06	Tratamiento duplicado	API responde 409	Mensaje: Ya existe un tratamiento con ese nombre	Automatizado aprobado
TRA-CP07	Agregar equipo	idTratamiento 9, idEquipo 2	POST /9/equipos	Automatizado aprobado
TRA-CP08	Remover equipo	idTratamiento 9, idEquipo 2	DELETE /9/equipos/2	Automatizado aprobado

Casos Por Valores Límite

Módulo	Campo	Dato	Escenario	Resultado esperado
Paciente	Documento	5 dígitos	Límite inferior -1	Error
Paciente	Documento	6 dígitos	Límite inferior exacto	OK
Paciente	Documento	15 dígitos	Límite superior exacto	OK
Paciente	Documento	16 dígitos	Límite superior +1	Error
Terapeuta	Usuario	3 caracteres	Límite inferior -1	Error
Terapeuta	Usuario	4 caracteres	Límite inferior exacto	OK
Terapeuta	Contraseña	5 caracteres	Límite inferior -1	Error
Terapeuta	Contraseña	6 caracteres	Límite inferior exacto	OK
Tratamiento	Duración	4	Límite inferior -1	Error
Tratamiento	Duración	5	Límite inferior exacto	OK
Tratamiento	Duración	240	Límite superior exacto	OK
Tratamiento	Duración	241	Límite superior +1	Error
Tratamiento	Sesiones	0	Límite inferior -1	Error
Tratamiento	Sesiones	1	Límite inferior exacto	OK
Tratamiento	Sesiones	50	Límite superior exacto	OK
Tratamiento	Sesiones	51	Límite superior +1	Error
Tratamiento	Costo	-1	Límite inferior -1	Error
Tratamiento	Costo	0	Límite inferior exacto	OK

Pruebas Del Camino Básico

Camino	Flujo	Secuencia	Resultado esperado
PAC-C1	Crear paciente exitoso	Formulario válido -> crearPaciente -> /Register -> respuesta OK	Paciente guardado y lista recargada
PAC-C2	Paciente duplicado	Formulario válido -> /Register -> 409	Mensaje de documento duplicado
PAC-C3	Consultar no paciente	obtenerPaciente -> API rol 2	Error: no es un paciente
TER-C1	Crear terapeuta exitoso	Formulario válido -> crearTerapeuta -> /Register -> respuesta OK	Terapeuta guardado con rol 2
TER-C2	Editar terapeuta	Formulario edición -> actualizarTerapeuta -> /Update/{id}	Terapeuta actualizado preservando rol 2
TER-C3	Consultar no terapeuta	obtenerTerapeuta -> API rol 1	Error: no es un terapeuta
TRA-C1	Crear tratamiento exitoso	Formulario válido -> crearTratamiento -> endpoint base	Tratamiento guardado
TRA-C2	Especialidad inválida	Formulario idEspecialidad = 0	No se envía; se muestra error
TRA-C3	Equipo excede disponibilidad	Cantidad requerida > disponible	Sistema pide confirmación antes de continuar
TRA-C4	Tratamiento duplicado	crearTratamiento -> 409	Mensaje funcional de duplicado

Ejecución Y Evaluación

Herramienta	Comando	Resultado
Vitest	npm run test	Test Files: 10 passed. Tests: 107 passed. Duration: 6.84 s.
Jest	npm run test:jest	Test Suites: 10 passed. Tests: 107 passed. Time: 7.041 s.
Vitest Coverage	npm run test:coverage	Statements 71.83%, Branches 70.70%, Functions 76.66%, Lines 71.75%.

Pruebas específicas de los tres módulos:

- pacientesService.test.js: 12 aprobadas.
 - terapeutasService.test.js: 12 aprobadas.
 - tratamientosService.test.js: 9 aprobadas.
- Total específico: 33 aprobadas.

Suite frontend completa:

- Vitest: 10 archivos aprobados, 107 pruebas aprobadas.
- Jest: 10 suites aprobadas, 107 pruebas aprobadas.

2.4 Pruebas De Integración

La integración se evalúa desde los formularios/listados hacia servicios y API client. En el nivel automatizado se prueban principalmente los servicios, y en el nivel de diseño se documentan los puntos de integración visual pendientes de ejecución manual.

ID	Integración	Entrada	Resultado esperado	Estado
INT-PAC-01	PatientForm -> pacientesService -> pacientesApi	Datos válidos	Payload incluye user/password igual a documento, activo true y fechaRegistro	Automatizado en servicio
INT-PAC-02	PatientList -> listarPacientes	API retorna mezcla de roles	Solo rol 1 aparece como paciente	Automatizado
INT-TER-01	TerapeutaForm -> terapeutasService -> terapeutasApi	Datos válidos	Payload preserva rol 2, activo true y fechaRegistro	Automatizado en servicio
INT-TER-02	TerapeutasList -> obtenerTerapeutasActivos	Terapeutas activos/inactivos	Solo activo true	Automatizado
INT-TRA-01	TratamientoForm -> tratamientosService	Datos válidos	Crea/actualiza tratamiento con especialidad, duración, costo, frecuencia y equipos	Automatizado parcial
INT-TRA-02	TratamientoForm -> listarEquipos/listarTratamientos	Equipo seleccionado	Calcula disponibilidad real según equipos ya usados	Diseñado/revisión de código
INT-TRA-03	TratamientosList -> listarTratamientos/listarEspecialidades	Carga de pantalla	Muestra tabla, métricas y buscador por nombre/descripción/especialidad	Diseñado/revisión de código

2.5 Pruebas De Sistema

Tipo	Foco	Herramienta	Resultado
Rendimiento	Carga y construcción de los tres módulos	npm run build	Build exitoso; advertencia general por chunk JS grande
Seguridad	Operaciones CRUD bajo cliente API autenticado	apiCliente + token JWT	Authorization Bearer centralizado; requiere sesión válida
Usabilidad	Formularios de pacientes, terapeutas y tratamientos	Revisión funcional	Campos requeridos, mensajes y estados de carga documentados
Portabilidad	Ejecución frontend en ambiente local	Node/Vite/Windows	Pruebas y build ejecutados correctamente

Build de sistema:
npm run build
Resultado: build exitoso.
Observación: Vite reporta advertencia por chunk JavaScript grande; no bloquea la entrega, pero se recomienda code-splitting en una iteración posterior.

2.6 Pruebas De Aceptación

ID	Requisito	Escenario	Pasos	Resultado esperado	Resultado obtenido
CA-PAC-01	RF-PAC-01	Registrar paciente	Completar formulario de paciente con datos válidos y guardar	Paciente creado con rol 1	Cubierto por servicio; UI pendiente manual
CA-PAC-02	RF-PAC-02	Listar pacientes	Abrir módulo de pacientes	Solo se observan personas con rol Paciente	Automatizado
CA-TER-01	RF-TER-01	Registrar terapeuta	Completar acceso, personal, contacto y profesional	Terapeuta creado con rol 2	Cubierto por servicio; UI pendiente manual
CA-TER-02	RF-TER-05	Consultar terapeutas activos	Abrir listado con mezcla de estados	Solo activos en filtro correspondiente	Automatizado
CA-TRA-01	RF-TRA-01	Registrar tratamiento	Completar nombre, especialidad, duración, costo y guardar	Tratamiento creado	Cubierto por servicio; UI pendiente manual
CA-TRA-02	RF-TRA-04	Asociar equipo	Seleccionar equipo y cantidad	Equipo asociado a tratamiento	Automatizado servicio
CA-TRA-03	RF-TRA-05	Cantidad mayor a disponible	Ingresar cantidad superior a disponibilidad real	Sistema advierte y solicita confirmación	Diseñado

3. Conclusiones

1. El entregable queda corregido para trabajar únicamente los tres módulos del PDF original: Pacientes, Terapeutas y Terapias/Tratamientos.
2. Pacientes cuenta con 12 pruebas automatizadas que validan filtrado por rol 1, creación, actualización, eliminación y errores del API.
3. Terapeutas cuenta con 12 pruebas automatizadas que validan filtrado por rol 2, creación, actualización preservando rol y terapeutas activos.
4. Tratamientos ahora cuenta con 9 pruebas automatizadas nuevas que cubren CRUD, error 409 y asociación/remoción de equipos.
5. La suite frontend completa quedó aprobada en Vitest y Jest con 107 pruebas exitosas.
6. Las pruebas de aceptación de interfaz quedan diseñadas y listas para ejecutarse manualmente o automatizarse con Selenium/Katalon.
7. Quedan pendientes pantallazos, diagramas gráficos y herramientas externas como JMeter/OWASP ZAP si el docente las exige como evidencia visual.

4. Referencias Y Pendientes

- Plantilla proyecto final - Pruebas.doc.
- Proyecto final primera entrega.pdf original, con los módulos de pacientes, terapeutas y tratamientos.
- Código fuente local de KineSys: kinesys/src/services y kinesys/src/pages/GestionAdmin.
- Salidas de ejecución en artifacts/pruebas: vitest-output-focused.txt, jest-output-focused.txt y coverage-output-focused.txt.

Pendiente	Motivo	Avance actual
Pantallazos de los tres módulos	No se levantó servidor visual durante esta corrección	Queda estructura lista para insertar capturas
Diagramas gráficos UML	No existen imágenes editables en el repositorio	Se dejan casos de uso, componentes y relaciones en tablas
Pruebas E2E Selenium/Katalon	No están configuradas	Casos de aceptación diseñados por módulo
JMeter/OWASP ZAP	No instalados	Se documenta build, seguridad de cliente API y pruebas unitarias/integración